

Arkitektur principper i praksis

Opgavesæt M11.01 - Kubernetes Volumes

Formål

I denne opgave skal I lave en deployment af en simpel container som kun kører en reduceret mængde af standard Linux værktøjerne, som f.eks. Bash.

I starter med at lave et deployment med 2 pods baseret på det samme container-image, hvor der anvendes en volumen med kortvarig (ephemeral) persistens.

Herefter laves et nyt deployment som anvender et volumen med varig persistens som er delt imellem de 2 pod-instanser.

I begge udgaver skal I prøve at nedlægge jeres pods for at simulere et crash og observere hvad der sker med jeres data på volumenerne når pods'ene genskabes.

Opgave A - Kortvarig volume

I denne opgave skal I oprette et deployment af 2 pods som anvender volumens med kortvarig persistens.

1. Start med gennemgå og deploy'e følgende `yaml`-fil til jeres lokale K8s-klynge:

```

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: kortvarig-app
5  spec:
6    replicas: 2
7    selector:
8      matchLabels:
9        app: kortvarig-app
10   template:
11     metadata:
12       labels:
13         app: kortvarig-app
14     spec:
15       containers:
16         - name: kortvarig-container
17           image: busybox:latest
18           command: ["/bin/sh", "-c", "sleep 3600"] # Holder containeren kørende
19           volumeMounts:
20             - name: ephemeral-opbevaring
21               mountPath: /data
22       volumes:
23         - name: ephemeral-opbevaring
24           emptyDir: {}

```

Gem den med navnet `ephemeral.yaml` og *deploy* den med kommandoen:

```
kubectl apply -f ephemeral.yaml
```

Bemærk: at der oprettes 2 instanser (se linje 6):

2. Tjek at der nu findes 2 nye pods i din klynge. Brug kommandoen `kubectl get pods`.
3. I en browser tilgå dit **K8s Dashboard** og åben en terminal ind i den ene pod. Det gør du ved at tilgå menuen `Workloads` -> `Pods`, og anvende `Exec`-menuen som du finder i punkt-menuen i højre side. Find folderen `/data` og observér at den er tom! Lad din terminal forblive åben.

4. Du skal nu kopiere en fil fra dit udviklingsmiljø ind i den **pod** som du har en terminal åben i. Brug den følgende kommando og anvend f.eks. den fil du lige har brugt til deployment og navnet på pod'en:

```
$ kubectl cp ephemeral.yaml kortvarig-app-64b466f5f6-fvc74:/data
```

Bemærk: første led efter `kubectl cp` er **kilden** (den lokale fil) og andet led er **destinationen** for kopi-kommandoen (pod-navn + stien inde i pod'en).

5. Gå tilbage til terminalen i browseren og tjek at filen nu ligger inde på volumen inde i klyngen.
6. Fra en terminal i din browser tilgå `/data` -folderen på den anden **kortvarig-app**-pod. Tjek at den ikke indholder noget data – altså de 2 pods deler ikke den samme volume og samme filsystem.
7. Slet nu den pod som har en fil i `/data` -folder (fra punkt-menuen i pods). Kubernetes vil nu genoprette en ny pod for at vedligeholde de 2 instanser som er beskrevet i deployment-spec'en. Når den kommer op igen, tilgå den nye pod og verificér at `/data` -folderen er tom – altså at volumener forsvinder når pod'en genoprettes!

Opgave B - Varig persistent volumen

Vi gentager opgave A, men denne gang bruger vi et **PersistentVolumeClaim** til at oprette et volumen som er **delt imellem de 2 pods**, og som **overlever genstart** af pods.

1. Start med deployment af følgende `yaml` -fil som opretter et **PersistentVolumeClaim**:

```
1 | apiVersion: v1
2 | kind: PersistentVolumeClaim
3 | metadata:
4 |   name: mit-sqlite-pvc
5 | spec:
6 |   accessModes:
7 |     - ReadWriteOnce
8 |   resources:
9 |     requests:
10 |       storage: 1Gi
```

2. Brug dit favorit søge-værktøj og undersøg hvad betyder `accessModes:` og den specifikke valgte `ReadWriteOnce` -mode? Hvad har den af betydning når vi anvender flere pods som tilgår den samme volume?

3. I dit K8s Dashboard find dit Persistent Volume Claim under **Config and Storage**. Og verificér under **Resource information** at dit claim er korrekt som defineret i trin 1 herover.
4. Opret nu 2 pods som anvender og deler volumen oprettet i trin 1. Brug følgende `yaml` -fil:

```

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: sqlite-app
5  spec:
6    replicas: 2
7    selector:
8      matchLabels:
9        app: sqlite-app
10   template:
11     metadata:
12       labels:
13         app: sqlite-app
14     spec:
15       containers:
16         - name: sqlite-container
17           image: busybox:latest
18           command: ["/bin/sh", "-c", "sleep 3600"] # Holder containeren kørende
19           volumeMounts:
20             - name: sqlite-data
21               mountPath: /data
22       volumes:
23         - name: sqlite-data
24           persistentVolumeClaim:
25             claimName: mit-sqlite-pvc

```

Bemærk ændringerne fra opgave A – den volumen som vi opretter peger på jeres **PersistentVolumeClaim** fra trin 1.

5. Ligesom i opgave A, tilgå `/data` -folderen og tjek at den er tom. Du kan bruge den ene eller begge `sqlite-app` -pods til formålet.
6. Kopiér nu en fil ind i den **ene** pod, som i trin 4 i opgave A. Og tjek at den samme fil er tilgængelig fra `/data` -folderen i begge pods.
7. Slet nu den ene pod, og vent på at den bliver genoprettet. Tilgå `/data` -folderen igen og tjek at filen er der stadigvæk.